

CSCE 2303 – Computer Organization and Assembly Language Programming Summer 2025

Project 1: ZX16 ISS

1. Background

An Instruction Set Architecture (ISA) Simulator (ISS) emulates a CPU by taking the machine code of a compiled or assembled program and executing its instructions as real hardware would. In this project, you will develop an ISS for the ZX16 Instruction Set. Inspired by the RISC-V architecture and based on the RISC principles, ZX16 is an open-source ISA developed at the AUC for educational purposes.

2. Project Requirements

A. Read and Understand the ZX16 ISA; the detailed ISA specifications can be found [here](#).

B. Develop a simulator for the ZX16 Instruction Set that meets the following specifications:

- Input Handling:
 - Read a ZX16 machine code file (.bin), which contains the binary-encoded instructions and data.
 - Assume the first instruction is located at memory address 0x0000.
- Instruction Processing:
 - You must support all true instructions
 - Decode each instruction word from the machine code.
 - Translate the decoded instruction into a human-readable string that represents the actual ZX16 instruction and print this string to the terminal.
 - Execute the instruction by updating registers, modifying memory, or performing I/O operations via ecall.
- Execution Environment:
 - The simulator must be executable from the command line using the following syntax:
`Zx16sim <machine_code_file_name>`
- Supported ecall Services:

Service Name	Service #	Description
Read String	1	Input: a0=address of the string buffer, a1=maximum length – Output: a0=string length
Read Integer	2	Output: a0=the read Integer
Print String	3	Input: a0=address of the string
Play tone	4	a0=frequency, a1=duration_ms
Set audio volume	5	a0=volume (0-255)
Stop Audio playback	6	
Read the keyboard	7	a0=the key code, a1=1 a key is pressed if, 0 nothing is pressed.
Registers Dump	8	Prints the values of all registers
Memory Dump	9	Prints the content of the memory from address in a0 for number of bytes in a1
Program Exit	10	

- You simulator should support 2D tiled graphics system similar to old videogame consoles.

2.1 The 2D Tiled Graphics System

2.1.1 Screen Resolution and Tiling Structure:

- Screen Resolution:** The display operates at a resolution of **320 pixels wide by 240 pixels high**. This is commonly known as QVGA (Quarter Video Graphics Array) resolution.
- Tile Dimensions:** The screen is not drawn pixel by pixel directly. Instead, it's composed of individual square "tiles." Each tile has a resolution of **16 pixels wide by 16 pixels high**.
- Screen as an Array of Tiles:** Given the screen and tile dimensions, the screen can be envisioned as a grid of tiles.
 - Horizontal tiles: 320 pixels / 16 pixels/tile = **20 tiles**
 - Vertical tiles: 240 pixels / 16 pixels/tile = **15 tiles**
 - Total tiles on screen: 20 tiles * 15 tiles = **300 tiles**

2.1.2 Memory Map for Graphics Data:

You will need to define specific memory regions for your graphics data within your simulator's memory space.

- Tile Map Buffer:**
 - Location:** This buffer starts at memory address 0xF000.
 - Purpose:** It holds the **array of tiles** that make up the current screen display. Think of it as a blueprint for what tiles appear where on the screen.

- **Structure:** It's a **row-major** array. This means that the tiles for the first row of the screen are stored sequentially, then the tiles for the second row, and so on.
- **Content:** Each entry, one byte, in this buffer will be a *tile number* (0 to 15, as defined below), indicating which of the 16 available tile definitions should be displayed at that position on the screen.
- **Tile Definitions (Tile Set/Tile Data):**
 - **Location:** The definitions for the 16 different tiles start at memory address 0xF200.
 - **Purpose:** This memory region stores the actual pixel data for each of the 16 unique tiles.
 - **Number of Tiles:** There are exactly **16 different tiles**, numbered from 0 to 15.
 - **Tile Data Representation:**
 - Each individual tile (16x16 pixels) is represented by the colors of its 256 pixels (16 rows * 16 columns).
 - **Pixel Color Depth:** Each pixel's color is specified using **4 bits**. This allows for $2^4=16$ unique colors per pixel.
 - **Tile Data Size:** Since each tile has 256 pixels, and each pixel needs 4 bits, a single tile requires $256 \text{ pixels} \times 4 \text{ bits/pixel} = 1024 \text{ bits}$.
 - **Byte Representation:** $1024 \text{ bits} / 8 \text{ bits/byte} = 128 \text{ bytes}$. Therefore, each tile definition occupies 128 bytes of memory.
 - **Packing:** The 4-bit pixel colors are packed into bytes.
 - The first byte has the color of pixel 0 (the lower 4 bits) of row 0 and pixel 1 (the high 4 bits) of row 0.
 - This packing continues: "The second byte has the colors of pixel 2 of row 0 and pixel 3 of row 0 and so on."
 - **Total Tile Definition Size:** $16 \text{ tiles} \times 128 \text{ bytes/tile} = 2048 \text{ bytes}$. This means the tile definitions will occupy memory from 0xF200 up to $0xF200 + 2048 - 1$.
- **Color Palette:**
 - **Location:** The color palette starts at memory address 0xFA00.
 - **Purpose:** This palette defines the actual RGB values for the 16 possible colors that can be used by the pixels.
 - **Structure:** It's an array of **16 bytes**. Each byte corresponds to one of the 16 colors (index 0 to 15).
 - **Color Encoding (within each byte):** Each byte defines a single color using an RGB format, with a specific bit allocation:
 - **Red (R):** Uses **3 bits** (bits 5, 6, and 7 of the byte).
 - **Green (G):** Uses **3 bits** (bits 2, 3, and 4 of the byte).
 - **Blue (B):** Uses **2 bits** (bits 0 and 1 of the byte).
 - **Total Palette Size:** 16 bytes. This means the palette will occupy memory from 0xFA00 up to $0xFA00 + 16 - 1$.

3. Development Guidelines

- Team Formation: Work in groups of three students.
- LLMs Usage: You may use LLM in development. However, you need to provide the prompt at the top of each file generated by LLM.
- Programming Language: A C++ skeleton is provided, and C/C++ is preferred; however, you may choose another programming language (e.g., Python) if you obtain prior approval from your instructor.
- Testing: Develop comprehensive test cases to validate the functionality and correctness of your simulator. At least 10 test cases exercising all instructions. An assembler is available in the ZX16 ISA GitHub repository to aid in creating test cases.
- Version Control and Workflow: Use GitHub for development. Maintain a disciplined workflow, as the commit history and individual contributions will be evaluated. Avoid informal methods (e.g., WhatsApp file sharing) for project development.
- Documentation: Your GitHub repository must include a readme file (in Markdown format) that serves as your project report. This document should detail:
 - Build Instructions: How to compile and build the simulator.
 - Usage Guidelines: How to run and use the simulator.
 - Design Overview: An explanation of your simulator's design and architecture.
 - Project Challenges: A discussion of any challenges faced and limitations in your implementation.

4. Testing Guidelines

Develop comprehensive test cases to validate the functionality and correctness of your simulator. All tests must be documented in your README file and organized systematically to ensure complete coverage of the ZX16 ISS functionality.

4.1 Test Documentation Requirements

Document each test case in your README file providing the following information:

- Test Case ID (serves as the test file name): Unique identifier (e.g., TC-ZX16-01)
- Test Objective: What specific functionality is being validated
- Test Steps: Sequential execution steps for running the test
- Expected Results: Specific register values, memory contents, or output strings

4.2 Test File Organization

- Test binaries should be named matching their Test Case ID (e.g., TC-ZX16-01.bin)
- Include corresponding assembly source files (e.g., TC-ZX16-01.s)
- Provide expected output files for comparison (e.g., TC-ZX16-01.expected)
- Store all test files in a dedicated tests/ directory within your repository

4.3 Required Test Coverage

Your test suite must comprehensively cover the following areas:

Instruction Categories: Data processing instructions (arithmetic, logical operations), Memory access instructions (load/store operations), and Control flow instructions (branches, jumps)

Ecall Services

Graphics system integration

4.4 Coverage Verification

You must provide a Test Coverage Table that lists all test cases and demonstrates complete requirement coverage. The coverage table must demonstrate that ALL requirements are tested with no gaps.

5. Grading Breakdown

- GitHub Development Compliance: -10% if GitHub is not used for development.
- Non-functional 2D Graphics: -10% (in addition to losing the 25% of the game).
- Non-Functional Audio: -5%
- Missing the deadline for group formation: -5%
- Readme File: 10% of the grade.
- Test Cases: 25% of the grade.
- Pong Game implemented in ZX16 assembly and runs on the simulator: 25%.
- Code Organization and Comments: 15% of the grade.
- Offline Test Case Results: 25% of the grade.

6. Timeline

- June 23: Group formation.
- June 30: First Milestone (at least 70% of the instructions are implemented, at least 2 ecalls are implemented, at least 3 test cases). Not delivering these will cause deductions of up to 30%
- July 9: Final Deliverables and interview.

7. References:

- ZX16 GH Repo: <https://github.com/shalan/ZX16>
- Commandline arguments in C: https://www.tutorialspoint.com/cprogramming/c_command_line_arguments.htm